МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-13

Выполнил:

Д. В. Шибун

студент группы ПО-13

Проверил:

А. Д. Кулик

13.02.2026

Брест 2026

Цель работы:

освоить возможности языка программирования Python в разработке оконных приложений

Задание 1. Построение графических примитивов и надписей

Код программы:

main.py — запуск приложения

```
from ui.window import AppWindow

if __name__ == "__main__":
    app = AppWindow()
    app.run()
```

ui/window.py — главное окно приложения

```
import tkinter as tk
from ui.controls import ControlsPanel
from graphics.animator import Animator
from graphics.moving_text import MovingText
from graphics.screenshot import make_screenshot


class AppWindow:
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Moving Text Animation")
        self.root.geometry("800x600")

        self.canvas = tk.Canvas(self.root, bg="black")
        self.canvas.pack(fill="both", expand=True)

        self.controls = ControlsPanel(self.root, self)

        self.objects = []
        self.animator = Animator(self)

        self.load_strings()

    def load_strings(self):
        for obj in self.objects:
            self.canvas.delete(obj.id)

        self.objects.clear()

        try:
            with open("data/strings.txt", "r", encoding="utf-8") as f:
                lines = [line.strip() for line in f.readlines() if line.strip()]
        except FileNotFoundError:
            lines = ["Hello", "Python", "Moving text"]

        for text in lines:
            obj = MovingText(self.canvas, text, self.controls.speed.get())
            self.objects.append(obj)

    def take_screenshot(self):
        make_screenshot(self.root)
```

```
    def run(self):
        self.animator.start()
        self.root.mainloop()
```

ui/controls.py — панель управления

```
import tkinter as tk


class ControlsPanel:
    def __init__(self, root, app):
        self.app = app

        frame = tk.Frame(root, bg="#222")
        frame.pack(fill="x")

        self.pause_btn = tk.Button(frame, text="Pause", command=self.toggle_pause)
        self.pause_btn.pack(side="left", padx=5, pady=5)

        self.speed = tk.DoubleVar(value=1.0)
        tk.Label(frame, text="Speed:", fg="white", bg="#222").pack(side="left")
        tk.Scale(
            frame, from_=0.1, to=5.0, resolution=0.1,
            orient="horizontal", variable=self.speed,
            command=self.update_speed
        ).pack(side="left")

        tk.Button(frame, text="Reload strings", command=self.app.load_strings).pack(side="left",
padx=5)
        tk.Button(frame, text="Screenshot", command=self.app.take_screenshot).pack(side="left",
padx=5)

    def toggle_pause(self):
        self.app.animator.paused = not self.app.animator.paused
        self.pause_btn.config(text="Resume" if self.app.animator.paused else "Pause")

    def update_speed(self, _):
        for obj in self.app.objects:
            obj.speed = self.speed.get()
```

graphics/moving_text.py — класс движущейся строки

```
import random


class MovingText:
    def __init__(self, canvas, text, speed):
        self.canvas = canvas
        self.text = text
        self.speed = speed

        w = canvas.winfo_width() or 800
        h = canvas.winfo_height() or 600

        self.x = random.randint(0, w)
        self.y = random.randint(0, h)
```

```
        self.dx = random.choice([-1, 1]) * random.uniform(1, 3)
        self.dy = random.choice([-1, 1]) * random.uniform(1, 3)

        self.id = canvas.create_text(self.x, self.y, text=text, fill="white", font=("Arial", 20))

    def move(self):
        self.canvas.move(self.id, self.dx * self.speed, self.dy * self.speed)
        self.x += self.dx * self.speed
        self.y += self.dy * self.speed

        w = self.canvas.winfo_width()
        h = self.canvas.winfo_height()

        if self.x < 0: self.x = w
        if self.x > w: self.x = 0
        if self.y < 0: self.y = h
        if self.y > h: self.y = 0

        self.canvas.coords(self.id, self.x, self.y)
```

graphics/animator.py — анимационный цикл

```
class Animator:
    def __init__(self, app):
        self.app = app
        self.paused = False

    def start(self):
        self.update()

    def update(self):
        if not self.paused:
            for obj in self.app.objects:
                obj.move()

        self.app.root.after(30, self.update)
```

graphics/screenshot.py — сохранение скриншота

```
from PIL import ImageGrab

def make_screenshot(root):
    x = root.winfo_rootx()
    y = root.winfo_rooty()
    w = root.winfo_width()
    h = root.winfo_height()

    img = ImageGrab.grab((x, y, x + w, y + h))
    img.save("screenshot.png")
```

data/strings.txt — входные данные

```
Hello world
Python is awesome
Moving text animation
Random direction
Dynamic scene
```

Спецификация ввода

Нет.

Пример

Нет.

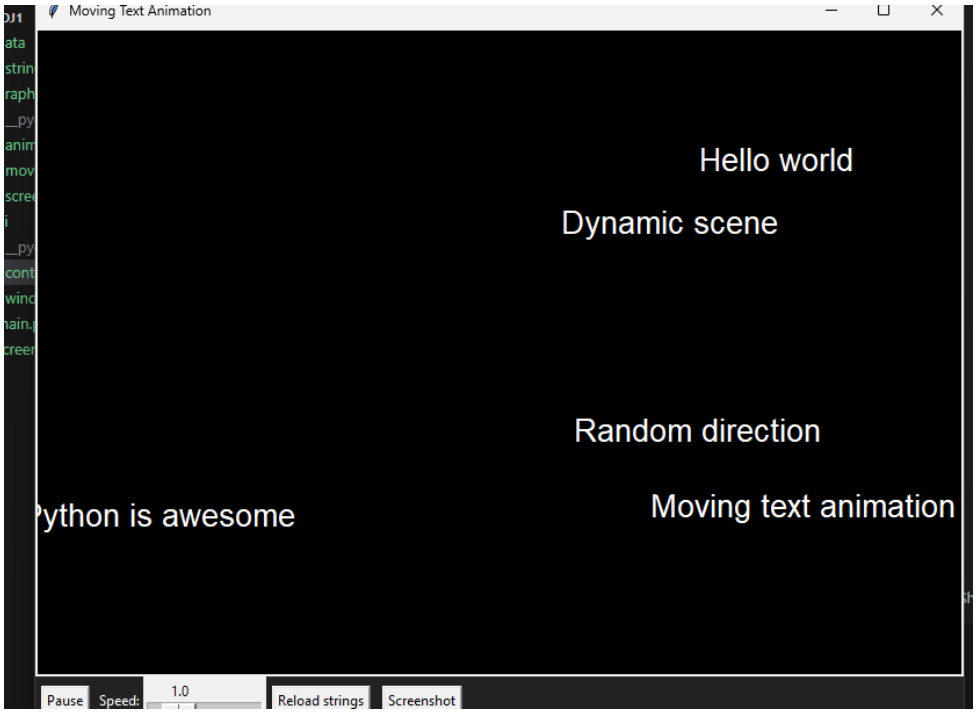
Спецификация вывода

Открывается графическое окно приложения.

Пример

python main.py

Рисунки с результатами работы программы



Задание 2. Реализовать построение заданного типа фрактала по варианту

Код программы:

main.py — запуск приложения

```
from ui.window import AppWindow

def main():

    app = AppWindow()

    app.run()

if __name__ == "__main__":

    main()
```

ui/window.py — главное окно приложения

```
import tkinter as tk
from ui.controls import ControlsPanel
from fractals.dragon import draw_dragon
from graphics.screenshot import make_screenshot
class AppWindow:
```

```
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Dragon Curve Fractal")
        self.root.geometry("900x700")

        self.canvas = tk.Canvas(self.root, bg="black")
        self.canvas.pack(fill="both", expand=True)

        self.controls = ControlsPanel(self.root, self)

    def build_fractal(self):
        iterations = self.controls.iter_var.get()
        length = self.controls.len_var.get()
        color = self.controls.color_var.get()
        self.canvas.delete("all")
        draw_dragon(self.canvas, iterations=iterations, length=length, color=color)

    def clear_canvas(self):
        self.canvas.delete("all")

    def take_screenshot(self):
        make_screenshot(self.root)

    def run(self):
        self.root.mainloop()
```

ui/controls.py — панель управления

```
import tkinter as tk
from tkinter import colorchooser

class ControlsPanel:
    def __init__(self, root, app):
        self.app = app

        frame = tk.Frame(root, bg="#222")
        frame.pack(fill="x")

        tk.Label(frame, text="Iterations:", fg="white", bg="#222").pack(side="left", padx=5)
        self.iter_var = tk.IntVar(value=12)
        tk.Entry(frame, textvariable=self.iter_var, width=5).pack(side="left")

        tk.Label(frame, text="Length:", fg="white", bg="#222").pack(side="left", padx=5)
        self.len_var = tk.IntVar(value=8)
        tk.Entry(frame, textvariable=self.len_var, width=5).pack(side="left")

        tk.Label(frame, text="Color:", fg="white", bg="#222").pack(side="left", padx=5)
        self.color_var = tk.StringVar(value="cyan")
        self.color_btn = tk.Button(frame, text="Choose", command=self.choose_color)
        self.color_btn.pack(side="left")

        tk.Button(frame, text="Build Dragon", command=self.app.build_fractal).pack(side="left",
padx=10)
        tk.Button(frame, text="Clear", command=self.app.clear_canvas).pack(side="left", padx=5)
        tk.Button(frame, text="Screenshot", command=self.app.take_screenshot).pack(side="left",
padx=5)
```

```
    def choose_color(self):
        color = colorchooser.askcolor(title="Choose color")[1]
        if color:
            self.color_var.set(color)
```

fractals/dragon.py — генерация и отрисовка кривой дракона

```
def generate_dragon_points(iterations, length):
    points = [(0.0, 0.0), (float(length), 0.0)]

    for _ in range(iterations):
        new_points = [points[0]]
        for i in range(len(points) - 1):
            x1, y1 = points[i]
            x2, y2 = points[i + 1]

            mx = (x1 + x2) / 2
            my = (y1 + y2) / 2

            dx = x2 - x1
            dy = y2 - y1

            rx = mx - dy / 2
            ry = my + dx / 2

            new_points.append((rx, ry))
            new_points.append((x2, y2))

        points = new_points

    return points
def draw_dragon(canvas, iterations=10, length=10, color="cyan"):
    canvas.update_idletasks()
    w = canvas.winfo_width() or 800
    h = canvas.winfo_height() or 600

    points = generate_dragon_points(iterations, length)

    xs = [p[0] for p in points]
    ys = [p[1] for p in points]
    min_x, max_x = min(xs), max(xs)
    min_y, max_y = min(ys), max(ys)

    offset_x = (w - (max_x - min_x)) / 2 - min_x
    offset_y = (h - (max_y - min_y)) / 2 - min_y

    shifted = [(x + offset_x, y + offset_y) for x, y in points]

    for i in range(len(shifted) - 1):
        x1, y1 = shifted[i]
        x2, y2 = shifted[i + 1]
        canvas.create_line(x1, y1, x2, y2, fill=color)
```

graphics/screenshot.py — сохранение скриншота

```
from PIL import ImageGrab
```

```
def make_screenshot(root, filename="dragon_screenshot.png"):
    root.update()
    x = root.winfo_rootx()
    y = root.winfo_rooty()
    w = root.winfo_width()
    h = root.winfo_height()

    img = ImageGrab.grab((x, y, x + w, y + h))
    img.save(filename)
```

Спецификация ввода

Нет.

Пример

Нет.

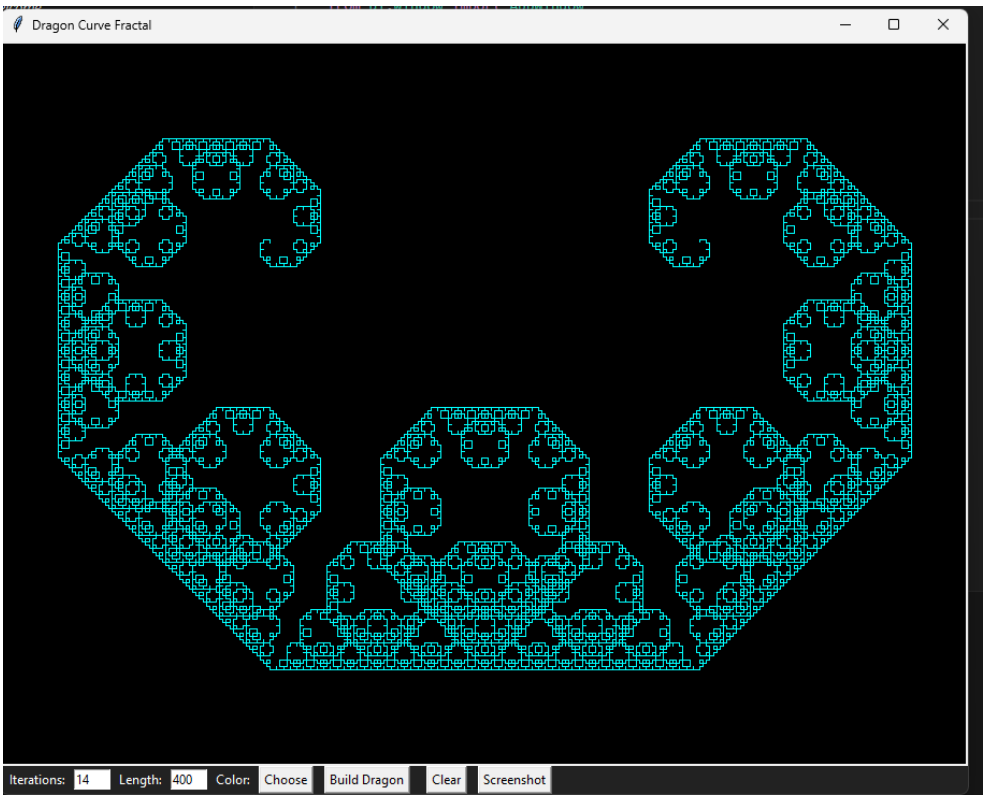
Спецификация вывода

Открывается графическое окно приложения.

Пример

python main.py

Рисунки с результатами работы программы



Вывод: В ходе выполнения работы был разработан программный комплекс для построения фрактала типа «Кривая дракона» с использованием графической библиотеки Tkinter. Проект реализован в модульной архитектуре, что позволило чётко разделить функциональность на компоненты: пользовательский интерфейс, генерацию фрактала, визуализацию и вспомогательные инструменты. Такой подход обеспечивает удобство сопровождения, расширяемость и возможность добавления новых типов фракталов в дальнейшем.